



Strathmore University

@iLabAfrica Centre

Session 3: Classical Machine Learning & Evaluation

Javier Serrano

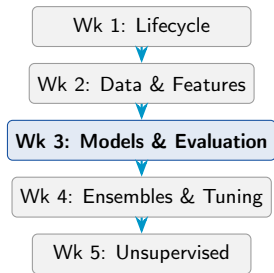
Applied Machine Learning

Master in Data Science & Analytics

Where we are in the course

From clean features to trustworthy models

- **Week 1:** the end-to-end ML lifecycle; framing, splits, baselines.
- **Week 2:** messy real-world data; feature engineering without leakage.
- **Week 3 (today):** train *classical* models and, above all, learn to **evaluate them rigorously**, the skill that separates a demo from a deployable system.
- Running thread: **thin-file mobile-money credit scoring**, a low-default-rate, high-cost-asymmetry problem where naïve evaluation is actively dangerous.



Learning objectives

By the end of today you will be able to...

1. **Derive and apply** the core classical learners, logistic regression, softmax regression, SVMs, k NN, naïve Bayes, and articulate when each is the right tool.
2. **Diagnose** why accuracy is a misleading metric under class imbalance, and select metrics (precision, recall, F_β , ROC-AUC, PR-AUC) aligned with business cost.
3. **Design** leakage-free evaluation protocols: stratified k -fold CV, pipelines fitted inside folds, and nested CV for model selection.
4. **Quantify uncertainty** in performance estimates (fold variance, bootstrap CIs) instead of reporting a single number.
5. **Handle imbalanced data** correctly: class weighting, threshold moving, and SMOTE applied *only* to training folds.
6. **Choose an operating threshold** for the mobile-money credit scorer by minimising expected business cost.

The Classical Toolbox

Threshold Curves & Calibration

Cross-Validation Done Right

Learning from Imbalanced Data

Case Study & Lab

Why classical ML still runs the world's tabular data

- Deep learning dominates perception (images, audio, text); on **tabular business data**, transactions, KYC records, repayment histories, linear models and trees remain state of practice.
- Reasons that matter *in production*:
 - **Latency & cost**: a logistic regression scores a loan application in microseconds on a CPU, critical for USSD-based flows on 2G networks.
 - **Interpretability & regulation**: the CBK and data-protection regimes increasingly expect *explainable* credit decisions; coefficients and odds ratios are auditable.
 - **Sample efficiency**: thin-file borrowers give you thousands, not millions, of rows.
 - **Strong baselines**: you cannot claim a transformer “works” until it beats a regularised logistic regression (Week 1’s baseline discipline).
- Today’s models are also the **vocabulary of evaluation**: every metric we build applies unchanged to XGBoost (Week 4) and neural nets (Week 6+).

Logistic regression: the model

A linear score pushed through a squashing function

- Linear score (log-odds / logit):

$$z = \boldsymbol{\theta}^\top \mathbf{x} + b$$

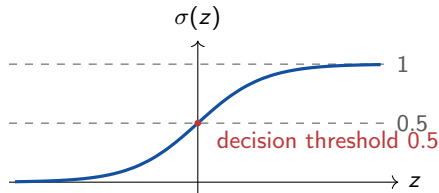
- Probability via the **sigmoid**:

$$\hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1)$$

- Equivalent statement: the model is linear in the *log-odds*,

$$\log \frac{\hat{p}}{1 - \hat{p}} = \boldsymbol{\theta}^\top \mathbf{x} + b$$

- So e^{θ_j} is an **odds ratio**, the auditable language a credit committee understands.



The 0.5 threshold is a *default*, not a law, we move it deliberately later today.

Logistic regression: training objective

Maximum likelihood $\Rightarrow$ cross-entropy loss

- Assume $y_i \in \{0, 1\}$ Bernoulli with parameter $\hat{p}_i = \sigma(\boldsymbol{\theta}^\top \mathbf{x}_i + b)$. Maximising the likelihood is minimising the **log loss** (binary cross-entropy):

$$J(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^m \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right]$$

- J is **convex**, gradient descent finds the global optimum, with the elegant gradient $\nabla_{\boldsymbol{\theta}} J = \frac{1}{m} \sum_i (\hat{p}_i - y_i) \mathbf{x}_i$.
- Regularisation** controls variance (Week 1's bias-variance trade-off):

$$J_{\ell_2} = J + \frac{\lambda}{2m} \|\boldsymbol{\theta}\|_2^2 \quad J_{\ell_1} = J + \frac{\lambda}{m} \|\boldsymbol{\theta}\|_1 \quad (\text{sparse; implicit feature selection})$$

- scikit-learn gotcha:** LogisticRegression regularises *by default* with $C = 1/\lambda$; larger C = weaker regularisation. Always treat C as a hyperparameter.

Multiclass: softmax regression

- For K classes, one score per class $z_k = \theta_k^\top x + b_k$, normalised by the **softmax**:

$$\hat{p}_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad \sum_{k=1}^K \hat{p}_k = 1$$

- Trained with categorical cross-entropy $J(\Theta) = -\frac{1}{m} \sum_i \sum_k y_{ik} \log \hat{p}_{ik}$.
- Prediction: $\hat{y} = \arg \max_k \hat{p}_k$. Decision boundaries between any two classes remain **linear**.
- Applied example: routing mobile-money customer complaints into {fraud dispute, failed transaction, KYC issue, other}, a 4-class problem with heavy imbalance across classes.
- This exact softmax layer reappears as the **output layer of every neural classifier** in Weeks 6–10; master it now.

Support vector machines: maximum margin

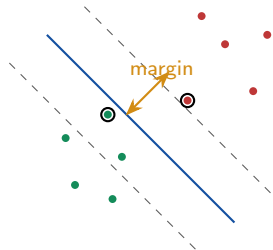
Hinge loss and the kernel trick

- Labels $y_i \in \{-1, +1\}$. The soft-margin SVM solves

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \max(0, 1 - y_i(w^\top x_i + b))$$

, the **hinge loss** term is zero beyond the margin, linear inside it.

- Only **support vectors** (points on or inside the margin) determine the boundary.
- Kernel trick**: replace inner products with $K(x, x')$, e.g. RBF $e^{-\gamma \|x - x'\|^2}$, nonlinear boundaries without computing ϕ explicitly.



Ringed points are support vectors; only they matter.

k NN and naïve Bayes: two more baselines

k -Nearest Neighbours

- Non-parametric: classify by majority vote among the k closest training points,

$$\hat{y} = \arg \max_c \sum_{i \in \mathcal{N}_k(x)} \mathbb{I}[y_i = c]$$

- No training, expensive prediction ($O(m)$ per query), the *opposite* of what production wants.
- Extremely sensitive to feature scaling (Week 2!) and to the curse of dimensionality.

Naïve Bayes

- Bayes' rule with a conditional-independence assumption:

$$\hat{y} = \arg \max_c P(c) \prod_{j=1}^n P(x_j | c)$$

- “Naïve” because features are rarely independent, yet it is a startlingly strong baseline for text (SMS-phishing / smishing detection).
- Trains in one pass; ideal first model for high-dimensional sparse data.

Looking ahead: decision trees get their own deep treatment inside ensembles, Week 4.

Code: a leakage-safe classical baseline

Everything lives inside a Pipeline, Week 2's discipline, operationalised

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression

num_cols = ["avg_wallet_balance", "txn_count_90d", "airtime_topup_freq", "tenure_months"]
cat_cols = ["region", "device_type", "agent_channel"]

pre = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                     ("sc", StandardScaler())]), num_cols),
    ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                     ("oh", OneHotEncoder(handle_unknown="ignore"))]), cat_cols),
])

clf = Pipeline([
    ("pre", pre),
    ("model", LogisticRegression(C=1.0, class_weight=None, max_iter=2000)),
])

clf.fit(X_train, y_train)           # transform statistics learned on TRAIN only
proba = clf.predict_proba(X_valid)[: , 1]  # scores, not labels -- thresholding comes later
```

Rigorous evaluation works on scores; thresholding is a separate, deliberate decision.

Roadmap

The Classical Toolbox

Beyond Accuracy: Metrics that Match the Problem

Threshold Curves & Calibration

Cross-Validation Done Right

Learning from Imbalanced Data

Case Study & Lab

The accuracy trap

Our credit-scoring dataset has a 3% default rate

- Consider the constant classifier `predict("repays")` for every applicant:

$$\text{Accuracy} = \frac{\text{correct}}{\text{total}} = 97\%$$

- A model that **never detects a single defaulter** looks excellent by accuracy. This is not a corner case, it is the *normal* situation in fraud, default, disease screening, and churn.
- The failure is structural: accuracy weights every error equally, but our errors have wildly different costs:
 - False negative** (approve a defaulter): lose the loan principal, e.g. KES 10,000.
 - False positive** (reject a good borrower): lose the margin, e.g. KES 800, *and* exclude a creditworthy person, the very failure financial inclusion aims to fix.
- Rule for this course: **never report accuracy alone** on an imbalanced problem. Always ask: *what is the base rate, and what does each error cost?*

The confusion matrix: the source of every metric

	pred. default	pred. repays
actual default	TP hit	FN miss
actual repays	FP false alarm	TN correct reject

- Convention: **positive class = the rare, costly event** (default). Fix this early; half of all metric bugs are a flipped positive label.
- Every threshold value produces a *different* confusion matrix from the same scores.
- The four cells are counts; all scalar metrics, precision, recall, specificity, F_β , MCC, are just ratios of these counts.
- In the lab you will print it first, always: `confusion_matrix(y_true, y_pred)`.

Precision, recall, and the F_β family

- **Precision**, of those we flagged as defaulters, how many truly were?

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall** (sensitivity, TPR), of the true defaulters, how many did we catch?

$$\text{Recall} = \frac{TP}{TP + FN}$$

- They trade off through the threshold: raise it $\Rightarrow$ precision $\uparrow$, recall $\downarrow$.
- F_β blends them, weighting recall β times more than precision:

$$F_\beta = (1 + \beta^2) \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \text{Precision} + \text{Recall}}, \quad F_1 = \frac{2PR}{P + R}$$

- Choose β from the *cost structure*, not habit: default detection (FN $\gg$ FP cost) $\Rightarrow$ $\beta = 2$; blocking legitimate M-Pesa transactions (FP is customer-hostile) $\Rightarrow$ $\beta = 0.5$.

From metrics to money: cost-sensitive evaluation

- Attach a cost to each cell of the confusion matrix and minimise **expected cost** instead of maximising an abstract score:

$$\mathbb{E}[\text{cost}] = c_{FN} FN + c_{FP} FP$$

(correct decisions cost 0 here;
generalise as needed).

- With calibrated probabilities $\hat{p}$, decision theory gives the optimal threshold in closed form:

$$t^* = \frac{c_{FP}}{c_{FP} + c_{FN}}$$

Mobile-money micro-loan example

	pred. default	pred. repays
actual default	0	$c_{FN} = 10,000$
actual repays	$c_{FP} = 800$	0

$$t^* = \frac{800}{800 + 10,000} \approx 0.074$$

We should flag anyone with $\geq 7.4\%$ default probability, far below the default 0.5 threshold. This is why *threshold choice is a business decision*, not a library default.

Code: the metrics workbench

```
import numpy as np
from sklearn.metrics import (confusion_matrix, classification_report,
                             precision_recall_fscore_support, fbeta_score)

y_pred = (proba >= 0.074).astype(int)      # cost-optimal threshold, NOT 0.5

print(confusion_matrix(y_valid, y_pred))   # always look at raw counts first
print(classification_report(y_valid, y_pred,
                           target_names=["repays", "default"], digits=3))

# Recall-weighted score for our FN-costly problem
f2 = fbeta_score(y_valid, y_pred, beta=2.0)

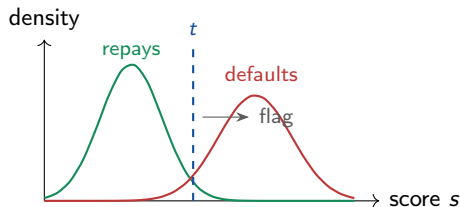
# Expected cost per 1,000 applications -- the number the CFO understands
tn, fp, fn, tp = confusion_matrix(y_valid, y_pred).ravel()
cost = 10_000 * fn + 800 * fp
print(f"F2 = {f2:.3f} | expected portfolio cost = KES {cost:,}")
```

Report the metric *and* the money. A model review meeting at a Nairobi fintech will ask for the second number.

Case Study & Lab

Scores, not labels: the threshold as a free parameter

- A classifier really outputs a **score** $s(x) \in [0, 1]$; sliding a threshold t over it sweeps out a whole *family* of classifiers.
- Curve-based evaluation asks: *how good is the score function across all thresholds?*
- Two canonical views:
 - **ROC**: TPR vs FPR as t varies.
 - **PR**: precision vs recall as t varies.



Moving t left catches more defaulters (recall $\uparrow$) at the price of more false alarms.

The ROC curve and AUC

- Plot, for every threshold:

$$\text{TPR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN}$$

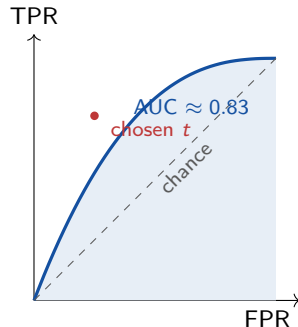
- AUC** = area under this curve.

Probabilistic meaning:

$$\text{AUC} = P(s(x^+) > s(x^-))$$

, the chance a random defaulter scores above a random repayer.

- 0.5 = coin flip; 1.0 = perfect ranking.
AUC is **threshold-free** and insensitive to class balance, both a strength and a blind spot.

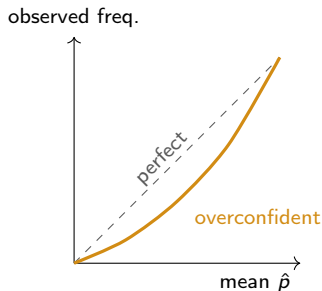


The precision–recall curve: the imbalance-honest view

- Under heavy imbalance, ROC can look flattering: FPR's denominator ($FP + TN$) is dominated by the huge negative class, so *thousands* of false positives barely move FPR.
- Precision, by contrast, is computed only over your *alerts*, it collapses immediately when false positives flood in. Davis & Goadrich (2006): a curve dominates in ROC space iff it dominates in PR space, but **differences are far more visible in PR space** when positives are rare.
- Baseline matters: a random classifier's PR curve sits at precision = prevalence (≈ 0.03 for us), not at 0.5. Judge PR-AUC (average precision) against *that*.
- Practical rule for this course:
 - Balanced classes, care about both error types $\Rightarrow$ **ROC-AUC**.
 - Rare positive class, care about alert quality (fraud, default, disease) $\Rightarrow$ **PR-AUC** + report the operating point.

Calibration: do the probabilities mean anything?

- **Calibrated** = among cases scored $\hat{p} \approx 0.2$, about 20% actually default. Our cost-optimal t^* *assumed* this!
- Diagnose with a **reliability diagram** (binned observed frequency vs mean $\hat{p}$); summarise with the **Brier score**
$$BS = \frac{1}{m} \sum_i (\hat{p}_i - y_i)^2.$$
- Repair: **Platt scaling** or **isotonic regression**, fitted on a held-out split, never on training data.
- Logistic regression is usually well calibrated; SVMs and boosted trees typically are not.



An overconfident scorer misprices micro-loan risk even if its *ranking* (AUC) is excellent.

Code: curves and calibration

```
from sklearn.metrics import (roc_curve, roc_auc_score,
                             precision_recall_curve, average_precision_score,
                             brier_score_loss)
from sklearn.calibration import CalibratedClassifierCV, calibration_curve

fpr, tpr, roc_thr = roc_curve(y_valid, proba)
prec, rec, pr_thr = precision_recall_curve(y_valid, proba)
print(f"ROC-AUC = {roc_auc_score(y_valid, proba):.3f}")
print(f"PR-AUC = {average_precision_score(y_valid, proba):.3f} (baseline = {y_valid.mean():.3f})")

# Reliability diagram data + Brier score
frac_pos, mean_pred = calibration_curve(y_valid, proba, n_bins=10, strategy="quantile")
print(f"Brier = {brier_score_loss(y_valid, proba):.4f}")

# Calibrate a poorly calibrated model (e.g., an SVM) with isotonic regression
calibrated = CalibratedClassifierCV(clf, method="isotonic", cv=5)
calibrated.fit(X_train, y_train)
```

strategy="quantile" keeps bins populated when positives are rare, uniform bins on a 3% base rate leave most bins nearly empty.

Case Study & Lab

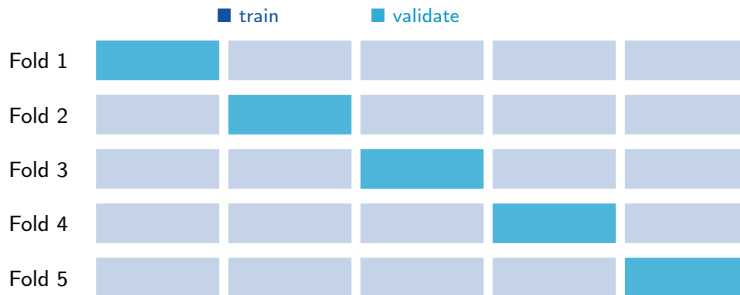
Why one train/validation split is not enough

- Any single split gives a performance estimate with **variance**: with only ~ 150 defaulters in a 5,000-row validation set, moving a handful of borrowers between splits can swing PR-AUC by several points.
- Decisions made on one split, “model A beats model B by 0.01 AUC”, are frequently just **split noise**.
- **k-fold cross-validation**: partition the training data into k folds; train on $k - 1$, validate on the held-out fold, rotate. Report

$$\hat{\mu} = \frac{1}{k} \sum_{j=1}^k \text{score}_j \quad \hat{\sigma} = \sqrt{\frac{1}{k-1} \sum_j (\text{score}_j - \hat{\mu})^2}$$

- House rule: a result reported **without a dispersion measure** ($\pm \hat{\sigma}$ or a CI) is incomplete, in this course and in your capstone.
- The **test set stays locked** until the very end (Week 1's contract). CV lives entirely inside the training data.

Stratified k -fold, visualised



- **Stratified** k -fold preserves the class ratio ($\approx 3\%$ defaulters) in every fold, mandatory under imbalance, otherwise some folds may contain almost no positives.
- Typical $k = 5$ or 10 : a bias–variance trade-off in the *estimator* itself (small k : pessimistic bias; large k : high variance, high cost).
- Time-correlated data (repayment histories, transactions) $\Rightarrow$ `TimeSeriesSplit`, never random folds, future information must not leak backwards (Week 2).

Leakage-safe CV and nested CV

The classic CV leak

- Scaling / imputing / resampling on the *full* training set **before** splitting into folds injects validation-fold statistics into training.
- Cure: cross-validate the **entire Pipeline**, so every transform is re-fitted inside each fold. In scikit-learn this is automatic if (and only if) preprocessing lives inside the Pipeline object.

Heuristic: anything learned from data, scaler means, encoder categories, SMOTE neighbours, selected features, tuned hyperparameters, must be learned inside the fold.

Nested CV for model selection

- Tuning hyperparameters *and* reporting performance on the same folds is optimistic: you selected the config that got lucky on those folds.
- **Nested CV**: inner loop selects hyperparameters; outer loop estimates generalisation of the whole *selection procedure*.
- Expensive but honest; Week 4 pairs it with Optuna for efficient search.

Is model A really better than model B?

Uncertainty and paired comparison

- Fold scores are **paired**: A and B see the same folds, so compare per-fold differences $d_j = \text{score}_j^A - \text{score}_j^B$, not two independent means.
- Quick screen: does the interval $\bar{d} \pm 2\hat{\sigma}_d/\sqrt{k}$ exclude 0? (Caution: fold scores are correlated because training sets overlap, so this test is approximate and *anti-conservative*, Dietterich 1998.)
- **Bootstrap CIs** on the held-out set: resample predictions with replacement $B \sim 1000$ times, recompute the metric, take the 2.5/97.5 percentiles, distribution-free and metric-agnostic.
- **McNemar's test** for two classifiers on one test set: uses only the discordant counts n_{01}, n_{10} (cases where exactly one model is right),

$$\chi^2 = \frac{(|n_{01} - n_{10}| - 1)^2}{n_{01} + n_{10}}$$

- Culture point: “+0.004 AUC” inside the noise band is not a result, it is a random seed

Code: cross-validating the full pipeline

```
import numpy as np
from sklearn.model_selection import StratifiedKFold, cross_validate, GridSearchCV

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

scores = cross_validate(
    clf, X_train, y_train, cv=cv,
    scoring=["roc_auc", "average_precision", "recall"], # multiple metrics at once
)
for m in ["test_roc_auc", "test_average_precision", "test_recall"]:
    print(f"{m}>26s: {scores[m].mean():.3f} +/- {scores[m].std():.3f}")

# Nested CV: inner loop tunes C, outer loop estimates generalisation honestly
inner = GridSearchCV(clf, {"model__C": np.logspace(-3, 2, 12)},
    scoring="average_precision",
    cv=StratifiedKFold(5, shuffle=True, random_state=0))
outer = cross_validate(inner, X_train, y_train, cv=cv, scoring="average_precision")
print(f"nested PR-AUC: {outer['test_score'].mean():.3f} +/- {outer['test_score'].std():.3f}")
```

Because `clf` is the whole Pipeline, imputation, scaling and encoding are re-fitted inside every fold, the leak is structurally impossible.

Roadmap

The Classical Toolbox

Beyond Accuracy: Metrics that Match the Problem

Threshold Curves & Calibration

Cross-Validation Done Right

Learning from Imbalanced Data

Case Study & Lab

Three families of remedies

Data level

Resample the training set:

- random under-sampling of majority
- random over-sampling of minority
- **SMOTE**: synthesise new minority points

Algorithm level

Re-weight the loss:

- `class_weight="balanced"` scales errors by inverse class frequency
- cost-sensitive losses (C_{FN} , C_{FP} in the objective)

Decision level

Keep data and loss as-is:

- **move the threshold** to the cost-optimal t^*
- often the simplest and best-performing fix

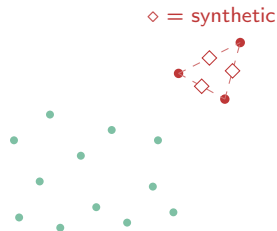
- Start at the **decision level**: it is free, reversible, and preserves calibration structure. Escalate to weighting, then to resampling, only if ranking quality itself is poor.
- Whatever you do to the data: **evaluate on the original, untouched distribution**, production traffic will have a 3% default rate, not 50%.

SMOTE: Synthetic Minority Over-sampling TEchnique

- For each minority point x_i : pick one of its k nearest *minority* neighbours x_{zi} , then interpolate:

$$x_{\text{new}} = x_i + \lambda (x_{zi} - x_i), \quad \lambda \sim \mathcal{U}(0, 1)$$

- Synthetic defaulters lie *on segments between real defaulters*, richer than duplicating rows.
- Caveats:
 - interpolation is dubious for categorical features (use SMOTE-NC);
 - can create points inside the majority region if classes overlap (Borderline-SMOTE, SMOTE+Tomek);
 - distorts calibration, recalibrate afterwards.



The cardinal sin: resampling before splitting

- If SMOTE runs on the **full dataset before** the train/validation split, synthetic points interpolated from validation-fold originals land in the training set, the model has effectively *seen* its validation data. Scores inflate dramatically and silently.
- Correct protocol: **SMOTE inside each CV fold, applied to the training portion only**; validation folds remain 100% real, original-distribution data.
- `imblearn.pipeline.Pipeline` enforces this automatically, its samplers act during `fit` but are bypassed at `predict` time. A plain `sklearn Pipeline` cannot hold a sampler; the `imblearn` variant exists precisely for this.
- Same logic extends Week 2's leakage doctrine: *resampling is a fitted transform; fitted transforms live inside folds*.
- This exact bug is one of the most common causes of “the model was great offline but collapsed in production” post-mortems in fintech ML teams.

Code: imbalance handled correctly

```

from imblearn.pipeline import Pipeline as ImbPipeline  # NOT sklearn's Pipeline
from imblearn.over_sampling import SMOTENC
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_validate

# Option 1: algorithm level -- reweight the loss, no data surgery
clf_w = clf.set_params(model=LogisticRegression(class_weight="balanced", max_iter=2000))

# Option 2: data level -- SMOTE-NC (mixed numeric/categorical), inside the pipeline
smote_clf = ImbPipeline([
    ("pre",  pre),                                     # ColumnTransformer from earlier
    ("smote", SMOTENC(categorical_features="auto", random_state=42)),
    ("model", LogisticRegression(max_iter=2000)),
])

# Both are cross-validated identically; SMOTE re-fits inside every fold,
# and validation folds stay at the true ~3% default rate.
for name, m in [("weighted", clf_w), ("smote", smote_clf)]:
    s = cross_validate(m, X_train, y_train, cv=cv, scoring="average_precision")
    print(f"{name:>9s}: PR-AUC = {s['test_score'].mean():.3f} +/- {s['test_score'].std():.3f}")

```

In the lab you will race options 1 and 2 against plain threshold-moving, and discover the simplest remedy is often competitive.

Roadmap

The Classical Toolbox

Beyond Accuracy: Metrics that Match the Problem

Threshold Curves & Calibration

Cross-Validation Done Right

Learning from Imbalanced Data

Case Study & Lab

Case study: evaluation design for the thin-file credit scorer

1. **Frame:** binary default prediction; positive class = default; prevalence $\approx 3\%$; costs $c_{FN} = \text{KES } 10,000$, $c_{FP} = \text{KES } 800$.
2. **Protocol:** temporal hold-out test set (last 3 months, locked); stratified 5-fold CV on the rest; entire preprocessing + model as one pipeline.
3. **Model ladder:** prevalence dummy $\rightarrow$ regularised logistic regression $\rightarrow$ + class weighting / SMOTE-NC $\rightarrow$ (Week 4) gradient boosting.
4. **Headline metric:** PR-AUC (imbalance-honest); **diagnostics:** ROC-AUC, reliability diagram, Brier score.
5. **Decision:** calibrate, then set $t^* = c_{FP} / (c_{FP} + c_{FN}) \approx 0.074$; report expected cost per 1,000 applications, with a bootstrap CI.
6. **Governance:** per-segment recall/precision (region, gender, device type), an accurate scorer that under-approves one county is a regulatory and ethical failure.
Looking ahead: drift monitoring of these same metrics, Week 13.

Practical session (1.5 h): the evaluation lab

Notebook: lab03_evaluation.ipynb, mobile-money loan dataset from Week 2, features already engineered.

1. **Baselines (15 min):** DummyClassifier (prevalence & stratified), record accuracy, PR-AUC. Observe the accuracy trap live.
2. **Classical ladder (25 min):** logistic regression, linear SVM, naïve Bayes in one Pipeline; stratified 5-fold CV; report mean $\pm$ std for ROC-AUC and PR-AUC.
3. **Curves (20 min):** plot ROC, PR and reliability diagrams for the best model; compute Brier score; calibrate with isotonic regression and re-plot.
4. **Imbalance shoot-out (20 min):** threshold-moving vs class_weight vs SMOTE-NC (imblearn pipeline). Which wins on PR-AUC? On expected cost?
5. **Decision memo (10 min):** choose an operating threshold from the cost matrix; write a 5-line justification a credit-risk officer could sign off.

Deliverable: pushed notebook + memo cell. Marking: correctness of CV protocol (40%), metric interpretation (30%), cost-based threshold argument (30%).

Key takeaways

- Classical models are production workhorses: fast, sample-efficient, auditable, and the mandatory baselines for everything that follows.
- Accuracy is meaningless at a 3% base rate; metrics must be chosen from the **cost structure** of the decision.
- Curves (ROC, PR) evaluate the *score function*; the operating **threshold** is a separate business decision, $t^* = C_{FP} / (C_{FP} + C_{FN})$ under calibration.
- **Calibration** turns scores into probabilities you can price risk with; check it, repair it.
- Cross-validate the **whole pipeline**; report mean $\pm$ dispersion; keep the test set locked; use nested CV when tuning.
- Imbalance remedies in order of preference: threshold moving $\rightarrow$ class weights $\rightarrow$ SMOTE (inside folds, evaluate on the true distribution).

Next week: ensembles and hyperparameter tuning, the same evaluation machinery, pointed at far stronger models.

Reading & references

Required

- Géron, *Hands-On Machine Learning*, 3rd ed.: Ch. 3 (Classification) and Ch. 4 (Training Models, logistic/softmax sections).
- Huyen, *Designing Machine Learning Systems*: Ch. 4 (class imbalance and evaluation sections).

Recommended

- Chawla et al. (2002), *SMOTE: Synthetic Minority Over-sampling Technique*, JAIR 16.
- Davis & Goadrich (2006), *The Relationship Between Precision-Recall and ROC Curves*, ICML.
- Dietterich (1998), *Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms*, Neural Computation.
- Niculescu-Mizil & Caruana (2005), *Predicting Good Probabilities with Supervised Learning*, ICML.